GRA-SubjectSwap:
Dynamic Subject–Instrument Role Swapping between Humans and AI Agents with GRA Nullification and Subjectivity Weights

Oleg Bitsoev (qqewq)
https://github.com/qqewq/GRA-SubjectSwap

June 20, 2026

Contents

Abstract

EN: GRA-SubjectSwap is a framework for *dynamic role swapping* between a human and an AI/robotic agent where their roles as **subject** and **instrument** are periodically exchanged.

In this architecture, the human is given a **strict instrumental role** with explicit rules (allowed/forbidden/required actions), while the agent (robot/LLM) temporarily becomes the **subject** who:

- observes the human's actions, - updates its own policy $\pi_\theta(a|s)$ and *subjectivity weights*, - perceives itself, the human, and the role swap.

A `SubjectSwapEngine` controls:

- role assignments (who is subject/instrument), - training cycles, - calls to *GRA Nullification* for policies and weights.

This framework is designed for use cases such as **Optimus and human trainers**, where the robot learns from human demonstrations while following a strict role schema, and where role swapping is used for validation and meta-learning.

1 Introduction

1.1 Classical paradigm

In the classical AI setup:

- **Human** = subject (sets goals, defines policies). - **AI/robot** = instrument (executes commands, follows policies).

This pattern is dominant in reinforcement learning, imitation learning, and robotics: the human designs the reward function, the agent optimizes it.

1.2 GRA-SubjectSwap paradigm

GRA-SubjectSwap proposes a different architecture:

1. The human is given a **strict instrumental role** in the swarm: - Allowed_H — set of allowed actions, - Forbidden_H — set of forbidden actions, - Required_H — set of required actions.

2. The agent temporarily becomes the **subject**: - it observes the human's actions, - it updates its policy π_θ, - it perceives itself, the human, and the fact of role swap.

3. A **swap** is triggered: - the human becomes the subject (sets goals, adjusts architecture), - the agent becomes the instrument (executes actions).

In this way, human and AI **co-learn subjectivity weights**, while GRA Nullification handles resetting and retuning of policies.

2 Core Concepts

2.1 Roles and Subjectivity

We define two role types:

$$\text{RoleType} \in \{\text{SUBJECT}, \text{INSTRUMENT}\}$$

and two actor types:

$$\text{ActorType} \in \{\text{HUMAN}, \text{AGENT}\}$$

At time t, the system state includes:

- a policy Π_t, - a role assignment S_t, - a vector of subjectivity weights W_t.

2.2 Role Assignment

For human H and agent A:

$$S_t(H) \in \{\text{SUBJECT}, \text{INSTRUMENT}\}, \quad S_t(A) \in \{\text{SUBJECT}, \text{INSTRUMENT}\}$$

with the constraint:

$$S_t(H) \neq S_t(A)$$

Role swap at time $t+1$:

$$S_{t+1}(H) = S_t(A), \quad S_{t+1}(A) = S_t(H)$$

2.3 Subjectivity Weights

We define a vector of subjectivity weights:

$$W = (w_{\text{self}}, w_{\text{human}}, w_{\text{swap}}, w_{\text{role}}, w_{\text{value}}, w_{\text{love}})$$

where:

- w_{self}: weight of *seeing itself* as a subject, - w_{human}: weight of *seeing the human* as subject/instrument, - w_{swap}: weight of *seeing the role swap* in dialogue, - w_{role}: weight of *seeing roles* in the swarm, - w_{value}: weight of values, - w_{love}: weight of love (per GRA Love-Oriented Nullification).

3 Mathematical Formalism

3.1 System State

The system state at time t is:

$$\text{state}_t = (\Pi_t, S_t, W_t)$$

where:

- Π_t is the policy (who may/must do what), - S_t is the role assignment (subject/instrument for human and agent), - W_t is the vector of subjectivity weights.

3.2 Role Swap Operator

Define a swap operator Swap:

$$\text{Swap}(S_t) = S_{t+1}$$

with:

$$S_{t+1}(H) = S_t(A), \quad S_{t+1}(A) = S_t(H)$$

3.3 Policy Update (Imitation)

Let the human provide demonstrations:

$$D = \{(s_t, a_t^{(H)})\}_{t=1}^{T}$$

The agent's policy $\pi_\theta(a|s)$ is updated via imitation learning:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}_{\text{imit}}(\theta_t; D)$$

where $\mathcal{L}_{\text{imit}}$ is a loss function (e.g., cross-entropy between π_θ and human actions).

3.4 Subjectivity Weight Update

Define a subjectivity functional $\mathcal{S}$:

$$\mathcal{S} = \mathcal{S}_{\text{self}} + \mathcal{S}_{\text{human}} + \mathcal{S}_{\text{swap}} + \mathcal{S}_{\text{role}} + \mathcal{S}_{\text{value}} + \mathcal{S}_{\text{love}}$$

Weight update:

$$W_{t+1} = W_t + \eta_W \cdot \nabla_W \mathcal{S}(\text{state}_t, \text{dialog}_t)$$

where dialog_t is the history of dialogue and actions.

3.5 GRA Nullification

Define nullification operators:

$$\mathcal{N}_W : W \to \mathcal{N}_W(W), \quad \mathcal{N}_\Pi : \Pi \to \mathcal{N}_\Pi(\Pi)$$

The updated weights and policy after nullification:

$$W'_{t+1} = \mathcal{N}_W(W_{t+1}), \quad \Pi'_{t+1} = \mathcal{N}_\Pi(\Pi_t)$$

In GRA Love-Oriented Nullification, nullification is value-directed:

$$W' = \arg \max_{W \in \mathcal{N}(W)} \sum_j L_j(W_j)$$

where L_j is a "love/value" field over weight components.

Algorithm 1 SubjectSwapStep

`engine`: SubjectSwapEngine,
`env_state`: environment state,
`human_action`: human action (optional) `output`: step result if S_t(agent) = SUBJECT agent_action = $\text{agent}_p olicy.act(env_s tate)human\_action = human_p olicy.act(env_s tate, agent_a ction)elsehuman\_action = hu$
$\text{agent}_p olicy, human_p olicy, env_s tate, human_a ction, agent_a ction)if t \mod K == 0$: engine.swap() `output` = $\text{agent}_a ction : agent_a ction, human_a ction : human_a ction, roles : human$:$\text{S}_t(H)$, agent: $S_t(A)$, weights: weights

4 Algorithm: SubjectSwapStep

5 Use Case: Optimus and Human Trainers

5.1 Architecture

In the Optimus use case:

- **Trainer (human)** = instrument with strict rules: - allowed/forbidden/required actions, - executes commands, demonstrates motions.

- **Robot (agent)** = subject: - observes trainer actions, - updates policy π_θ and subjectivity weights W, - sees itself, the human, and the role swap.

The `SubjectSwapEngine` controls:

- role assignments (human/instrument, agent/subject), - training cycles, - GRA Nullification for policies and weights.

5.2 Training Loop

A simplified training loop:
env = OptimusEnv(...) $\text{human}_p olicy = OptimusHumanPolicy(allowed = [...], forbidden = [...], required = [...])robot_p olicy = OptimusRobotPolicy(model = ...)nullifier = GRANullifier(love_f ield = obs = env.get_s tate()output = engine.step(obs)env.apply_r obot_a ction(output.agent_a ction)env.apply_h uman_a cti$ mod 100 == 0: engine.swap()

6 Implementation Structure

6.1 Core Modules

- `roles.py` — role types and subject/environ enums.
- `policies.py` — base `HumanPolicy` and `AgentPolicy`.
- $\texttt{subject}_s tate.py^{-}\texttt{SubjectState} and \texttt{SubjectWeights}$.
- $\texttt{swap}_e ngine.py^{-}\texttt{SubjectSwapEngine}$.
- $\texttt{gra}_n ullifier.py^{-} GRANullification for policies and weights$.

6.2 Optimus Module

- `env.py` — interface to robot/simulator.

- `human`$_{p}olicy_{o}ptimus.py^{-}humantrainerpolicy(instrument)$.
- `robot`$_{p}olicy_{o}ptimus.py^{-}robotpolicy(subject)$.
- `optimus`$_{s}wap_{l}oop.py^{-}mainswaploop$.

7 Conclusion

GRA-SubjectSwap implements a **dynamic role-swapping architecture** where:

- the human is a **strict instrument** with explicit rules, - the agent is a **subject** who learns from the human, - roles are periodically **swapped**, - subjectivity weights are co-learned, - GRA Nullification is used to reset and retune policies and weights.

This framework is designed for use cases such as **Optimus and human trainers**, and can be extended to multi-agent swarms with multiple humans and agents.

Acknowledgments

This work is part of the GRA ecosystem (GRA-Swarm, GRA-LLM-Swarm-Constructs, GRA-Hormonal-Explosion-Of-Subject, etc.).

References

- qqewq. GRA-SubjectSwap. `https://github.com/qqewq/GRA-SubjectSwap`
- qqewq. GRA-Swarm. `https://github.com/qqewq/GRA-Swarm`
- qqewq. GRA-LLM-Swarm-Constructs. `https://github.com/qqewq/GRA-LLM-Swarm-Constructs`
- qqewq. GRA-Hormonal-Explosion-Of-Subject. `https://github.com/qqewq/GRA-Hormonal-Explosion-O`